ARIMA-RF Model

May 25, 2026

1 Objectives

Import necessary libraries

Prepare data

- ☒ Import data

- ☒ Convert data to time series
- ☒ Split data to train/test

Data Analysis

- ☒ Plot data
- ☒ Hypothesis testing ADF test

SARIMA/ARIMA Modeling

- ☒ Residual testing
- ☒ Inverse AR/MA roots testing

Forecasting Data (using test_set)

- ☒ Forecast values using the length of the test set
- ☒ Plot the forecasted data
- ☒ Compare the actual test_data to forecasted_data
- ☒ Evaluate the model, check if it's okay to proceed

Preparation for Random Forest Training

- ☒ Extract the residuals
- ☒ Check for the significant lags for RF training
- ☒ Create dataframe for the residuals
- ☒ Fit the residuals to RF model
- ☒ Create multistep recursive prediction
- ☒ Create Final Prediction
- ☒ Plot the forecasted values
- ☒ Evaluate the model
- ☒ Compare to standalone ARIMA model

Using the model to whole data set

- ☒ Fit the whole data set to ARIMA model
- ☒ Check for the residual if it behaves as white noise
- ☒ Check for the inverse root of AR and MA
- ☒ Plot the standalone ARIMA forecast
- ☒ Extract the residuals
- ☒ Create dataframe for the extracted residuals
- ☒ Fit the dataframe to RF model
- ☒ Create recursive prediction
- ☒ Predict the residuals
- ☒ Create the final model
- ☒ Plot the forecasted values

Importing Necessary Libraries

[25]:
```
# For data manipulation and Math
import pandas as pd
import numpy as np

# For SARIMA modeling
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.graphics.tsaplots import plot_predict
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import pacf
import pmdarima as pm
import warnings

# For Machine Learning (Random-Forest)
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error,␣
 ↪root_mean_squared_error

# Visualizations
import matplotlib.pyplot as plt
import seaborn as sns
```

Data Preparations

[26]:
```
# Importing the Dataset as df
df = pd.read_excel("Inflation rates from bsp.xlsx")

# Converting the dataset as timeseries
df['month'] = pd.to_datetime(df['month'])
#setting the date as its index
df.set_index('month', inplace = True)
```

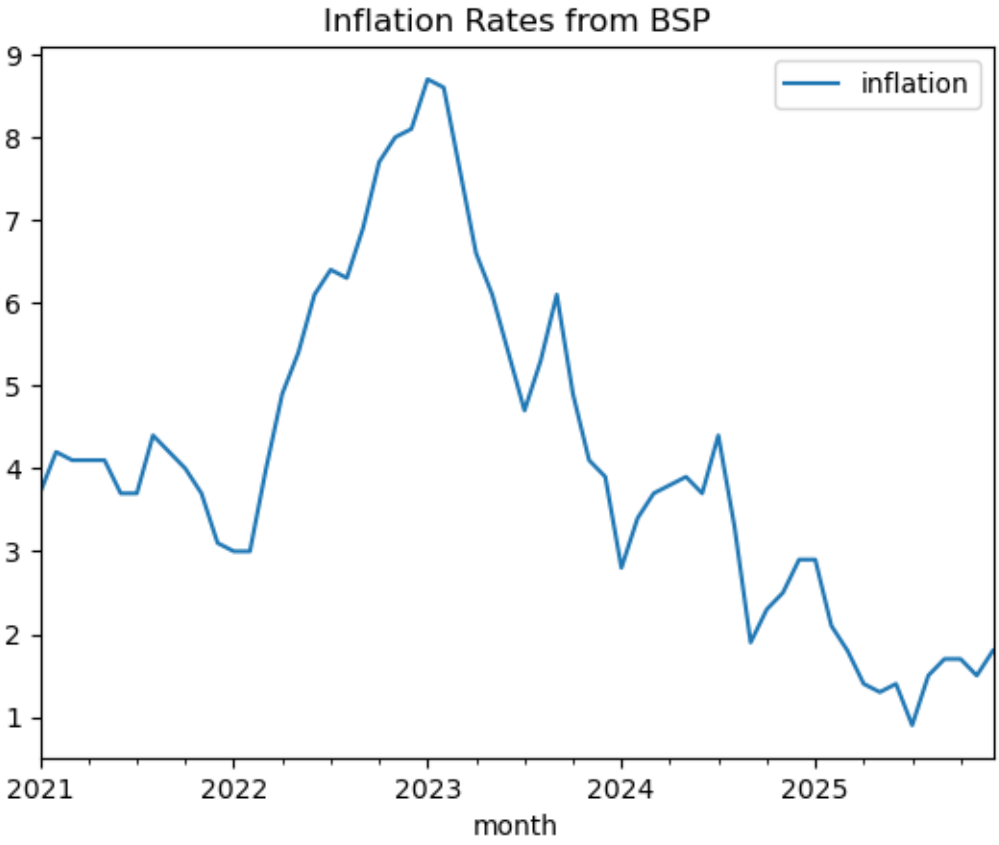
```
#Settubg the Frequency to monthly
df = df.resample('M').mean()

# Splitting the data into Train/Test sets
## Getting the split_index
split_index = int(len(df)*0.7)
## Getting the Train and Test Set
df_train = df.iloc[:split_index]
df_test = df.iloc[split_index:]
#Checking the length of each sets
print(f'Train set length: {len(df_train)}')
print(f'Test set length: {len(df_test)}')
```

```
Train set length: 42
Test set length: 18
```

Exploratory Data Analysis, meeting the Assumptions for SARIMA modeling

[27]:
```
# Plotting the Whole dataset to see how data move through time
df.plot(title = 'Inflation Rates from BSP')
```

[27]: <Axes: title={'center': 'Inflation Rates from BSP'}, xlabel='month'>

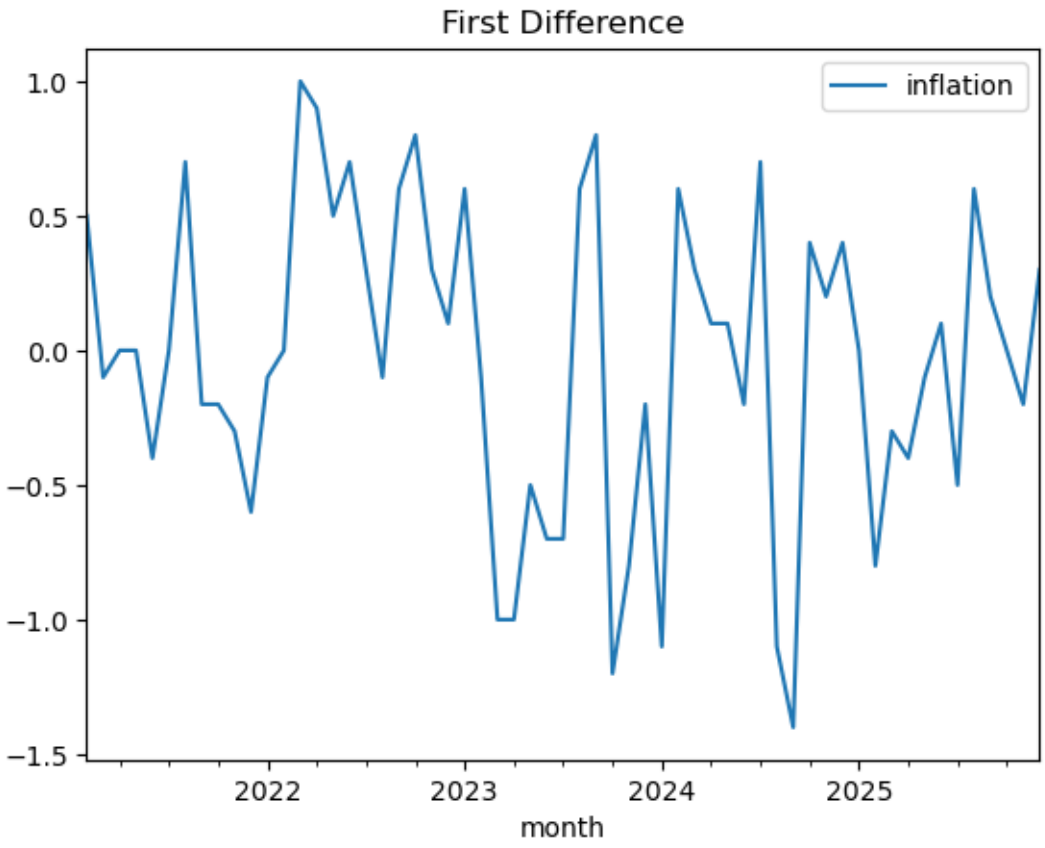


[28]:
```
# ADF test to see if the data is stationary, if not the data needs differencing
def adf_test(series):
    result = adfuller(series.dropna())
    print(f'ADF statistic: {result[0]}')
    print(f'p-value: {result[1]}')

    if result [1] <= 0.05:
        print(f'Conclusion: Stationary (Reject H0)')
        print(f'Proceed to modelling')
    else:
        print(f'Conclusion: Non-stationary (Fail to reject H0)')
        print(f'Proceed to apply differencing')
adf_test(df)
```

```
ADF statistic: -1.9284294532999926
p-value: 0.3188074298937776
Conclusion: Non-stationary (Fail to reject H0)
Proceed to apply differencing
```

[29]:
```
# First Difference and plot of the data
ts_diff = df.diff().dropna()
ts_diff.plot(title = 'First Difference')

# ADF testing the differenced data
adf_test(ts_diff)
```

```
ADF statistic: -2.97063078339282
p-value: 0.0377296379759372
Conclusion: Stationary (Reject H0)
Proceed to modelling
```



Creating SARIMA Model

[86]:
```
# Fitting the parameters to initiate the auto fit of SARIMA model
with warnings.catch_warnings():
    warnings.simplefilter('ignore')
    stand_model1 = pm.auto_arima(
        df_train['inflation'],
        seasonal = False,
        m = 12,
        d = 1,
        D = 0,
        start_p = 0, start_q = 0,
        max_p = 3, max_q = 3,
        start_P = 0, start_Q = 0,
        max_P =2, max_Q = 2,
        trace = True,
        error_action = 'ignore',
        supress_warnings = True,
        stepwise = True)
```

```
stand_model1_order = stand_model1.order
```

```
Performing stepwise search to minimize aic
 ARIMA(0,1,0)(0,0,0)[0] intercept   : AIC=75.230, Time=0.02 sec
 ARIMA(1,1,0)(0,0,0)[0] intercept   : AIC=70.958, Time=0.03 sec
 ARIMA(0,1,1)(0,0,0)[0] intercept   : AIC=68.109, Time=0.03 sec
 ARIMA(0,1,0)(0,0,0)[0]             : AIC=73.230, Time=0.01 sec
 ARIMA(1,1,1)(0,0,0)[0] intercept   : AIC=inf, Time=0.10 sec
 ARIMA(0,1,2)(0,0,0)[0] intercept   : AIC=68.925, Time=0.05 sec
 ARIMA(1,1,2)(0,0,0)[0] intercept   : AIC=inf, Time=0.10 sec
 ARIMA(0,1,1)(0,0,0)[0]             : AIC=66.111, Time=0.01 sec
 ARIMA(1,1,1)(0,0,0)[0]             : AIC=inf, Time=0.12 sec
 ARIMA(0,1,2)(0,0,0)[0]             : AIC=66.929, Time=0.03 sec
 ARIMA(1,1,0)(0,0,0)[0]             : AIC=68.959, Time=0.02 sec
 ARIMA(1,1,2)(0,0,0)[0]             : AIC=inf, Time=0.09 sec

Best model:  ARIMA(0,1,1)(0,0,0)[0]
Total fit time: 0.608 seconds
```

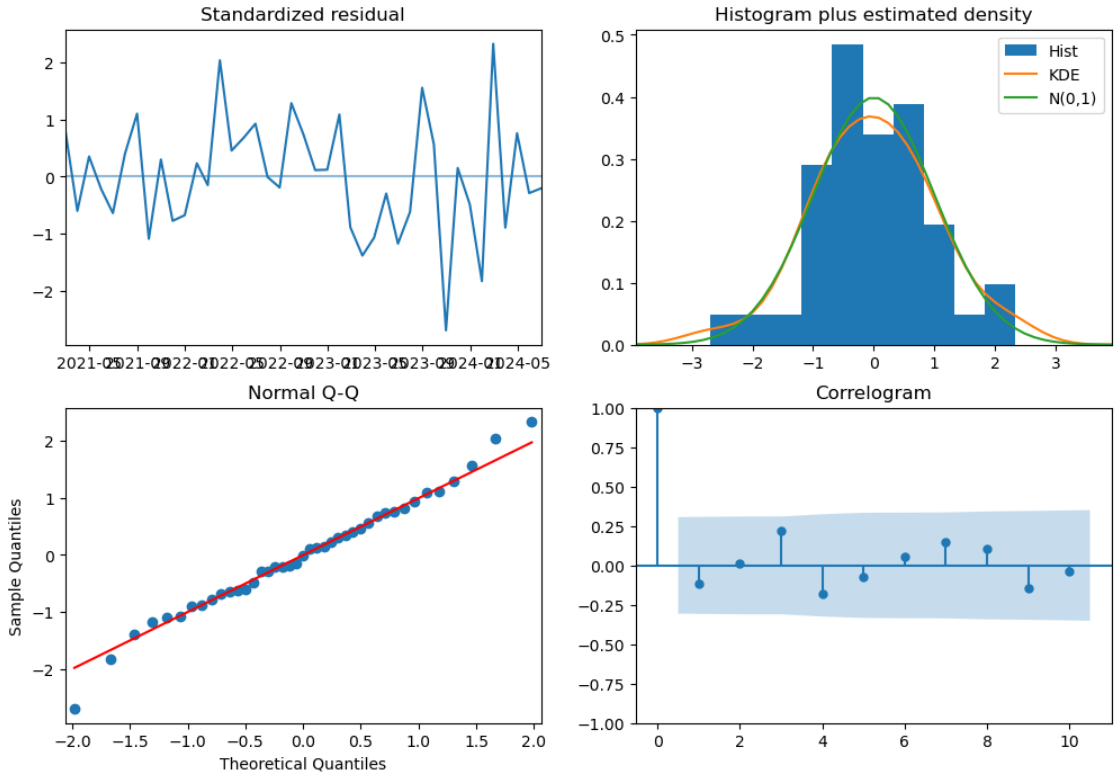
[31]:
```
# Viewing the model Summary
print(stand_model1.summary())
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                      y   No. Observations:                   42
Model:               SARIMAX(0, 1, 1)   Log Likelihood                -31.056
Date:                Sun, 24 May 2026   AIC                             66.111
Time:                        22:32:18   BIC                             69.538
Sample:                  01-31-2021   HQIC                            67.359
                       - 06-30-2024
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ma.L1          0.6342      0.158      4.021      0.000       0.325       0.943
sigma2         0.2630      0.054      4.856      0.000       0.157       0.369
===================================================================================
===
Ljung-Box (L1) (Q):                   0.57   Jarque-Bera (JB):
0.21
Prob(Q):                              0.45   Prob(JB):
0.90
Heteroskedasticity (H):               2.34   Skew:
-0.06
Prob(H) (two-sided):                  0.12   Kurtosis:
3.33
===================================================================================
===
```

```
Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-
step).
```

[32]:
```
# Checking residuals (Assumption no.2)
stand_model1.plot_diagnostics(figsize=(12,8))     #We can see in the plot 4 of␣
 ↪them satisfies that the residual behaves as white noise

#further check for residuals (Ljung-Box test)
lb_test = acorr_ljungbox(stand_model1.resid(), lags=[10], return_df=True)
print(lb_test)
```

```
     lb_stat  lb_pvalue
10  3.675329   0.960807
```



We can see from above that the residuals of the model we got behaves as white noise

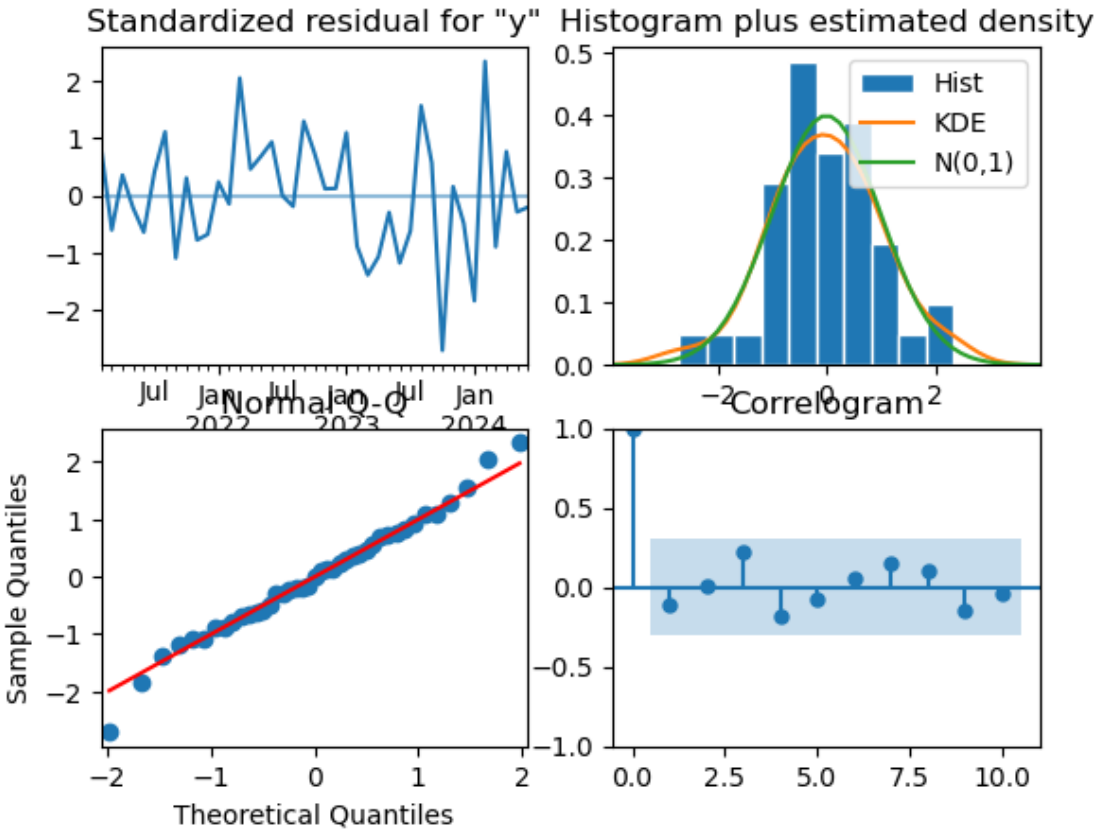
We can see from Histogram that the distribution follows bell curve (normal distribution)

We can see from Normal Q-Q plot that the plot follows straight line as what we want it to be

Lastly, we can see that there are no random lags that exceed our confidence bounts on our correlogram

This satisfies our first assumption that the residuals must behave as white noise

[33]:
```
stand_model1.arima_res_.plot_diagnostics()
plt.show()
```



Checking for Reverse AR roots for our second Assumption

[34]:
```
def inv_roots(model):
    inv_ar_roots = 1 / model.arroots()
    inv_ma_roots = 1/ model.maroots()

    print(f'MA roots invertible: {np.all(np.abs(inv_ma_roots) < 1)}')
    print(f'AR roots invertible: {np.all(np.abs(inv_ar_roots) < 1)}')

inv_roots(stand_model1)
```

```
MA roots invertible: True
AR roots invertible: True
```

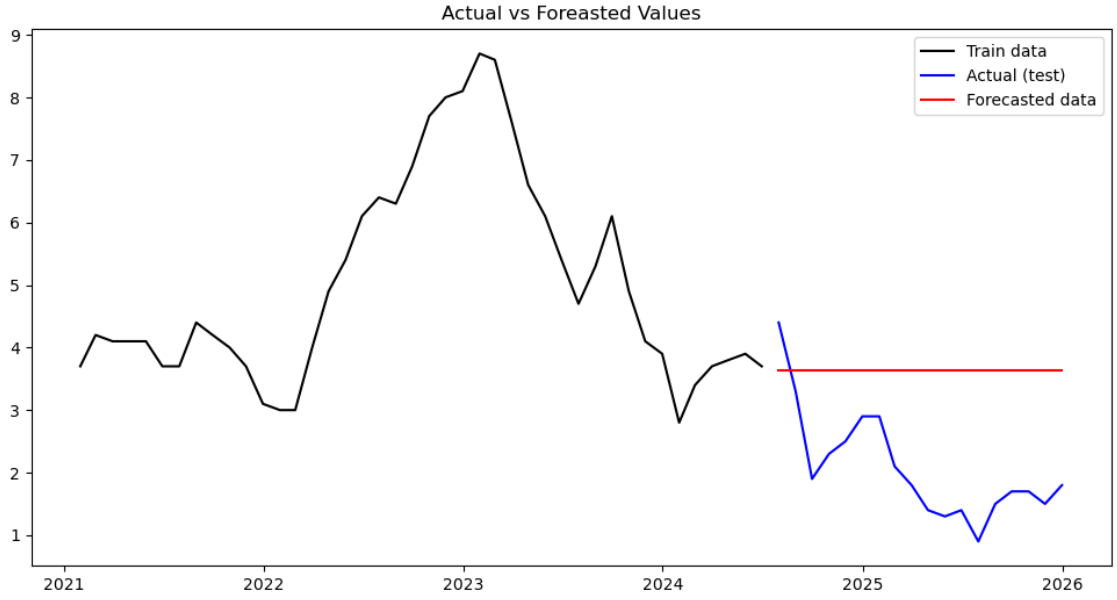
Forecasting using the length of df_test

[35]:
```
with warnings.catch_warnings():
    warnings.simplefilter('ignore')
```

```
    stand_forecast_1 = stand_model1.predict(n_periods=len(df_test))
```

Plot the df_train along with forecasted values

[36]:
```
plt.figure(figsize = (12,6))

plt.plot(df_train.index, df_train, label  = 'Train data', color = 'black')
plt.plot(df_test.index, df_test, label = 'Actual (test)', color = 'blue')
plt.plot(df_test.index,stand_forecast_1, label = 'Forecasted data', color =␣
 ↪'red')

plt.title('Actual vs Foreasted Values')
plt.legend()
plt.show()
```



Getting the error metrics to evaluate the model

[37]:
```
def err_metrics(y_true, y_pred):
    # 1. Mean Squared Error (MSE)
    mse = mean_squared_error(y_true, y_pred)

    # 2. Root Mean Squared Error (RMSE)
    # In newer versions of sklearn, you can use squared=False
    rmse = root_mean_squared_error(y_true, y_pred)

    # 3. Mean Absolute Error (MAE)
    mae = mean_absolute_error(y_true, y_pred)
```

```
    print(f"MSE: {mse}")
    print(f"RMSE: {rmse}")
    print(f"MAE: {mae}")

err_metrics(df_test, stand_forecast_1)
```

```
MSE: 3.1305007701179104
RMSE: 1.7693221216380894
MAE: 1.645753375200405
```

Checking if the error is good in respect to value of the actual data

[38]:
```
def scale(y_true, y_pred):
    data_range = np.max(y_true) - np.min(y_true)
    nrmse = root_mean_squared_error(y_true, y_pred) / data_range
    err_as_per_range = round(nrmse * 100, 2)

    print(f"Error as percentage of range: {nrmse * 100:.2f}%")
    if err_as_per_range == 0 and err_as_per_range <= 10.00:
        print(f'model is excellent')
    elif err_as_per_range > 10 and err_as_per_range <= 25:
        print(f'Model is Good/Fair')
    else:
        print('Model is poor: further fine tuning is required')


scale(df_test, stand_forecast_1)
```

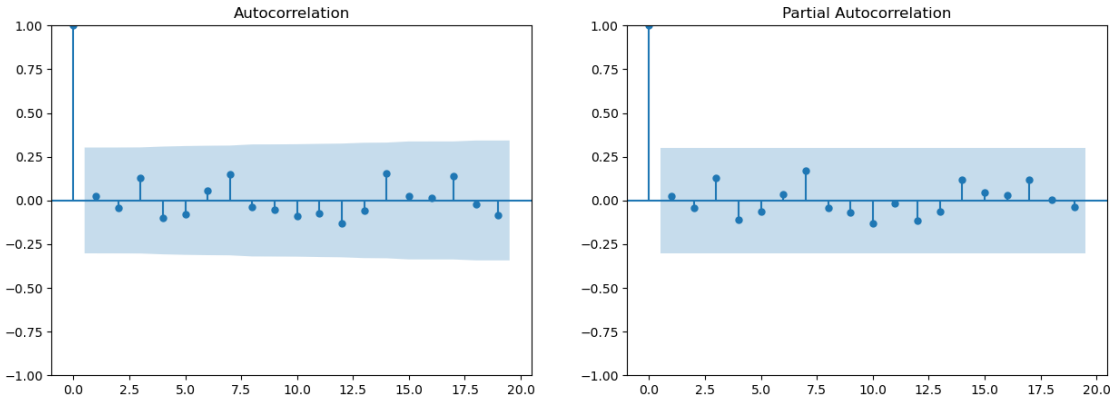
```
Error as percentage of range: 50.55%
Model is poor: further fine tuning is required
```

Data Preparation for Random Forest training

[39]:
```
# Extracting Residuals
residuals = stand_model1.resid()
```

Checking for significant lag for RF training

[40]:
```
fig, ax = plt.subplots(1, 2, figsize = (15, 5))

plot_acf(residuals, ax = ax[0], lags = 19)
plot_pacf(residuals, ax = ax[1], lags = 19)
plt.show()
```



Extracting the significant lag and creating data frame out of it

[41]:
```
# Selecting the significant lags to be used
sig_lags = [7,10]

# Creating Dataframe for lags chosen
def df_lag(data, lags):
    lag_df = pd.DataFrame(data, columns=['residual_target'])

    #Create lag Columns
    for lag in lags:
        lag_df[f'lag_{lag}'] = lag_df['residual_target'].shift(lag)

    lag_df =lag_df.dropna()

    return lag_df

resid_df = df_lag(residuals, sig_lags)
# Creating relationship between two lags
resid_df['diff'] = resid_df['lag_7'] - resid_df['lag_10']
```

Random Forest modeling

[42]:
```
X = resid_df[['lag_7', 'lag_10', 'diff']]
y = resid_df['residual_target']

rf_model1 = RandomForestRegressor(n_estimators = 500,
                                  max_depth = 4,
                                  min_samples_leaf = 5,
                                  random_state = 42
                                 )

rf_model1.fit(X,y)
```

[42]:
```
RandomForestRegressor(max_depth=4, min_samples_leaf=5, n_estimators=500,
                      random_state=42)
```

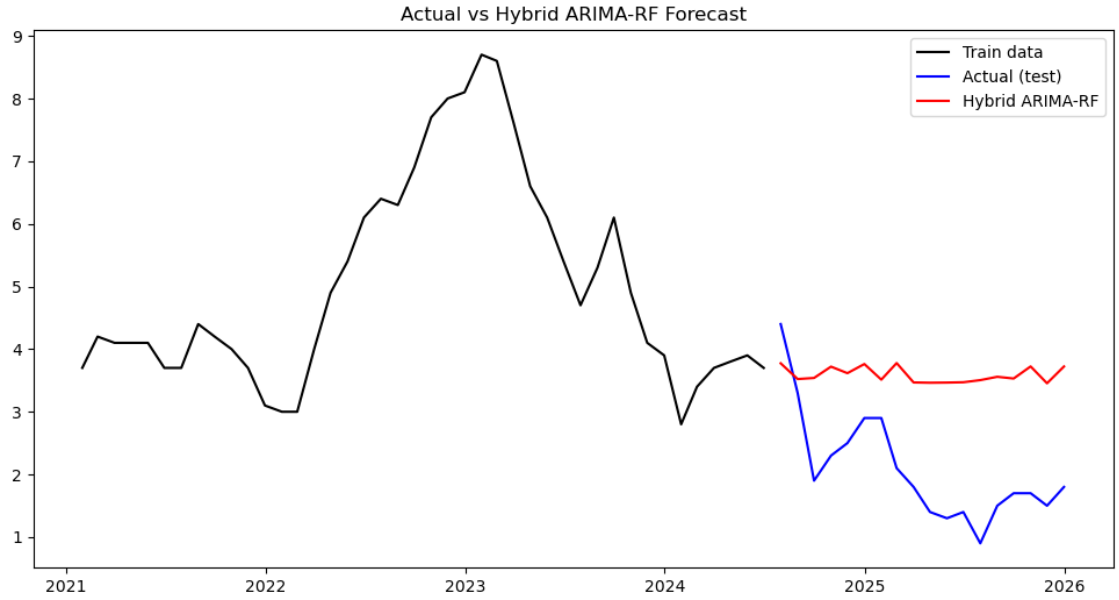
Multi Step Recursive Prediction

[43]:
```
# Initializing the variable needed
n_forecast = len(df_test)
predicted_residuals = []

# Getting the last known residual to start history
history_residuals = list(residuals[-max(sig_lags):])

# Recursice loop:
for i in range(n_forecast):
    input_row = pd.DataFrame({
        'lag_7' : [history_residuals[-7]],
        'lag_10' : [history_residuals[-10]],
        'diff' : [history_residuals[-10] - history_residuals[-7]]
    })

    # Predict Next 'correction' (residual)
    next_pred = rf_model1.predict(input_row)[0]

    # Store and add to history so next step can use it as a lag
    predicted_residuals.append(next_pred)
    history_residuals.append(next_pred)

# Final Result
predicted_residuals = np. array(predicted_residuals)
```

[44]:
```
# Prediction
final_hybrid_forecast1 = stand_forecast_1 + predicted_residuals
```

Plotting the Hybrid ARIMA-RF model Forecast

[45]:
```
plt.figure(figsize = (12,6))

plt.plot(df_train.index, df_train, label  = 'Train data', color = 'black')
plt.plot(df_test.index, df_test, label = 'Actual (test)', color = 'blue')
plt.plot(df_test.index,final_hybrid_forecast1, label = 'Hybrid ARIMA-RF', color␣
 ↪= 'red')

plt.title('Actual vs Hybrid ARIMA-RF Forecast')
plt.legend()
plt.show()
```



Evaluating the model

[46]:
```
def err_metrics(y_true, y_pred):
    # 1. Mean Squared Error (MSE)
    mse = mean_squared_error(y_true, y_pred)

    # 2. Root Mean Squared Error (RMSE)
    # In newer versions of sklearn, you can use squared=False
    rmse = root_mean_squared_error(y_true, y_pred)

    # 3. Mean Absolute Error (MAE)
    mae = mean_absolute_error(y_true, y_pred)

    print(f"MSE: {mse}")
    print(f"RMSE: {rmse}")
    print(f"MAE: {mae}")

print(f'Hybrid Model Metrics Evaluation: ')
err_metrics(df_test, final_hybrid_forecast1)
```

```
Hybrid Model Metrics Evaluation:
MSE: 2.9072624743218394
RMSE: 1.7050696391414162
MAE: 1.5855528258983318
```

Comparing the standalone model to hybrid model

[47]:
```
print(f'Standalone Model:')
train_stand_model = err_metrics(df_test, stand_forecast_1)
```

```
print(train_stand_model)

print(f'Hybrid Model:')
train_hybrid_model = err_metrics(df_test, final_hybrid_forecast1)
print(train_hybrid_model)
```

```
Standalone Model:
MSE: 3.1305007701179104
RMSE: 1.7693221216380894
MAE: 1.645753375200405
None
Hybrid Model:
MSE: 2.9072624743218394
RMSE: 1.7050696391414162
MAE: 1.5855528258983318
None
```

Hybrid Model Appeared to be better than the stand alone and that's what we want.

Will now proceed to fit the whole data set to the model

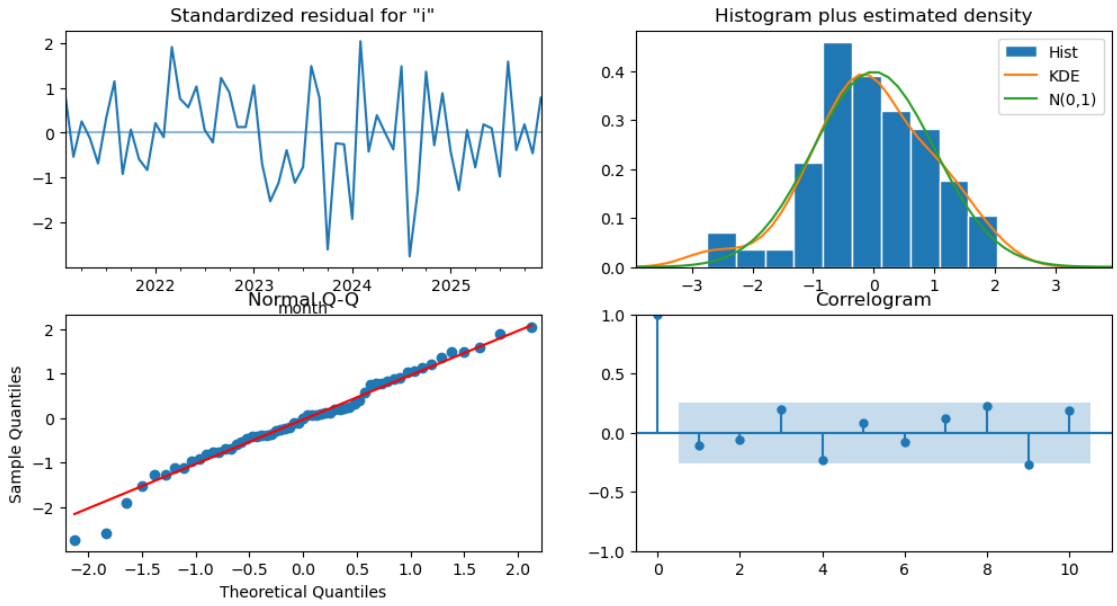
Refitting the full dataset to ARIMA model

[80]:
```
with warnings.catch_warnings():


    warnings.simplefilter('ignore')
    final_ARIMA1 = ARIMA(df['inflation'], order = stand_model1_order).fit()
```

Checking for the residuals

[81]:
```
final_ARIMA1.plot_diagnostics(figsize = (12,6))
test = acorr_ljungbox(final_ARIMA1.resid, lags = [10], return_df = True)
print(test)

# p-value is grated than 0.5, the residual behaves as white noise, we can␣
 ↪proceed to check for inverese AR / MA roots
```

```
     lb_stat  lb_pvalue
10  6.183489   0.799619
```



[82]:
```
# check for inverse roots of AR/MA
def inv_roots(model):
    inv_ar_roots = 1 / model.arroots
    inv_ma_roots = 1/ model.maroots

    print(f'MA roots invertible: {np.all(np.abs(inv_ma_roots) < 1)}')
    print(f'AR roots invertible: {np.all(np.abs(inv_ar_roots) < 1)}')

inv_roots(final_ARIMA1)

# both are invertible
# we now satisfied the 3 assumptions for the model
```

```
MA roots invertible: True
AR roots invertible: True
```

Final ARIMA forecast

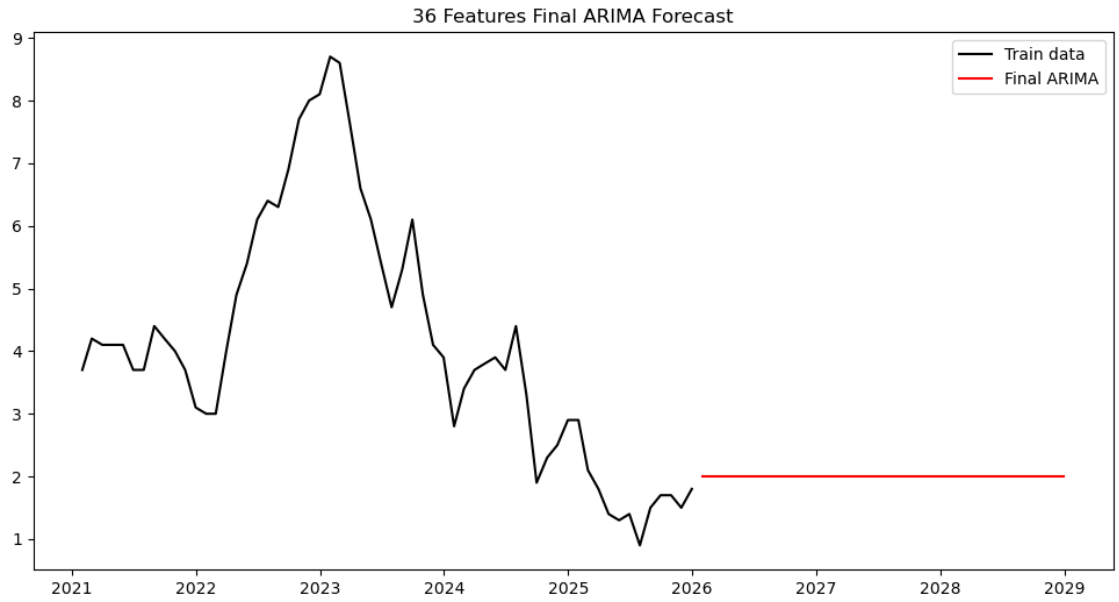
[83]:
```
with warnings.catch_warnings():
    warnings.simplefilter('ignore')
    final_arima_forecast = final_ARIMA1.forecast(36)

final_arima_forecast
final_arima_forecast
```

[83]:
```
2026-01-31    2.00106
2026-02-28    2.00106
2026-03-31    2.00106
```

```
2026-04-30    2.00106
2026-05-31    2.00106
2026-06-30    2.00106
2026-07-31    2.00106
2026-08-31    2.00106
2026-09-30    2.00106
2026-10-31    2.00106
2026-11-30    2.00106
2026-12-31    2.00106
2027-01-31    2.00106
2027-02-28    2.00106
2027-03-31    2.00106
2027-04-30    2.00106
2027-05-31    2.00106
2027-06-30    2.00106
2027-07-31    2.00106
2027-08-31    2.00106
2027-09-30    2.00106
2027-10-31    2.00106
2027-11-30    2.00106
2027-12-31    2.00106
2028-01-31    2.00106
2028-02-29    2.00106
2028-03-31    2.00106
2028-04-30    2.00106
2028-05-31    2.00106
2028-06-30    2.00106
2028-07-31    2.00106
2028-08-31    2.00106
2028-09-30    2.00106
2028-10-31    2.00106
2028-11-30    2.00106
2028-12-31    2.00106
Freq: M, Name: predicted_mean, dtype: float64
```

Plot the final ARIMA Forecast

[71]:
```
plt.figure(figsize = (12,6))

plt.plot(df.index, df, label  = 'Train data', color = 'black')
plt.plot(final_arima_forecast.index,final_arima_forecast, label = 'Final␣
 ↪ARIMA', color = 'red')

plt.title('36 Features Final ARIMA Forecast')
plt.legend()
plt.show()
```



Create dataframe/matrix using the residuals as target and lags as features

[73]:
```
# Extracting full residuals
full_resid = final_ARIMA1.resid

final_df = df_lag(full_resid, sig_lags)
final_df['diff'] = final_df['lag_10'] - final_df['lag_7']
final_df
```

[73]:

month	residual_target	lag_7	lag_10	diff
2021-11-30	-0.318025	0.135707	3.700000	3.564293
2021-12-31	-0.447404	-0.064502	0.499999	0.564501
2022-01-31	0.114675	-0.369117	-0.295013	0.074104
2022-02-28	-0.055024	0.177023	0.135707	-0.041316
2022-03-31	1.026402	0.615070	-0.064502	-0.679572
2022-04-30	0.407509	-0.495117	-0.369117	0.125999
2022-05-31	0.304468	0.037567	0.177023	0.139456
2022-06-30	0.553909	-0.318025	0.615070	0.933096
2022-07-31	0.034222	-0.447404	-0.495117	-0.047713
2022-08-31	-0.116420	0.114675	0.037567	-0.077108
2022-09-30	0.655861	-0.055024	-0.318025	-0.263002
2022-10-31	0.485303	1.026402	-0.447404	-1.473806
2022-11-30	0.067141	0.407509	0.114675	-0.292834
2022-12-31	0.067784	0.304468	-0.055024	-0.359491
2023-01-31	0.567475	0.553909	1.026402	0.472492
2023-02-28	-0.372288	0.034222	0.407509	0.373287

```
2023-03-31        -0.821368 -0.116420  0.304468  0.420888
2023-04-30        -0.605889  0.655861  0.553909 -0.101952
2023-05-31        -0.209281  0.485303  0.034222 -0.451081
2023-06-30        -0.599582  0.067141 -0.116420 -0.183561
2023-07-31        -0.412307  0.067784  0.655861  0.588077
2023-08-31         0.797834  0.567475  0.485303 -0.082173
2023-09-30         0.417181 -0.372288  0.067141  0.439428
2023-10-31        -1.400173 -0.821368  0.067784  0.889152
2023-11-30        -0.128165 -0.605889  0.567475  1.173364
2023-12-31        -0.138503 -0.209281 -0.372288 -0.163007
2024-01-31        -1.033543 -0.599582 -0.821368 -0.221786
2024-02-29         1.095917 -0.412307 -0.605889 -0.193582
2024-03-31        -0.225846  0.797834 -0.209281 -1.007115
2024-04-30         0.208366  0.417181 -0.599582 -1.016763
2024-05-31         0.000021 -1.400173 -0.412307  0.987866
2024-06-30        -0.200010 -0.128165  0.797834  0.925999
2024-07-31         0.795969 -0.138503  0.417181  0.555684
2024-08-31        -1.481924 -1.033543 -1.400173 -0.366630
2024-09-30        -0.688939  1.095917 -0.128165 -1.224083
2024-10-31         0.730569 -0.225846 -0.138503  0.087343
2024-11-30        -0.150543  0.208366 -1.033543 -1.241909
2024-12-31         0.472234  0.000021  1.095917  1.095896
2025-01-31        -0.226589 -0.200010 -0.225846 -0.025836
2025-02-28        -0.691278  0.795969  0.208366 -0.587603
2025-03-31         0.031691 -1.481924  0.000021  1.481945
2025-04-30        -0.415206 -0.688939 -0.200010  0.488929
2025-05-31         0.099225  0.730569  0.795969  0.065401
2025-06-30         0.052389 -0.150543 -1.481924 -1.331381
2025-07-31        -0.525138  0.472234 -0.688939 -1.161173
2025-08-31         0.851973 -0.226589  0.730569  0.957157
2025-09-30        -0.208796 -0.691278 -0.150543  0.540734
2025-10-31         0.100185  0.031691  0.472234  0.440543
2025-11-30        -0.248071 -0.415206 -0.226589  0.188617
2025-12-31         0.419030  0.099225 -0.691278 -0.790503
```

Fitting the final_df to RF

[87]:
```
final_X = final_df[['lag_7', 'lag_10', 'diff']]
final_y = final_df['residual_target']

rf_model2 = RandomForestRegressor(n_estimators = 500,
                                  max_depth = 10,
                                  min_samples_leaf = 2,
                                  random_state = 42
                                 )

rf_model2.fit(final_X,final_y)
```

[87]:
```
RandomForestRegressor(max_depth=10, min_samples_leaf=2, n_estimators=500,
                      random_state=42)
```

Creating Recursive Prediction

[112]:
```
# Initializing the variable needed
n_forecast = 36
predicted_residuals = []

# Getting the last known residual to start history
history_residuals = list(full_resid[-max(sig_lags):])

# Recursice loop:
for i in range(n_forecast):
    input_row = pd.DataFrame({
        'lag_7' : [history_residuals[-7]],
        'lag_10' : [history_residuals[-10]],
        'diff' : [history_residuals[-10] - history_residuals[-7]]
    })

    # Predict Next 'correction' (residual)
    next_pred = rf_model2.predict(input_row)[0]

    # Store and add to history so next step can use it as a lag
    predicted_residuals.append(next_pred)
    history_residuals.append(next_pred)

# Final Result
final_rf = np. array(predicted_residuals)
```

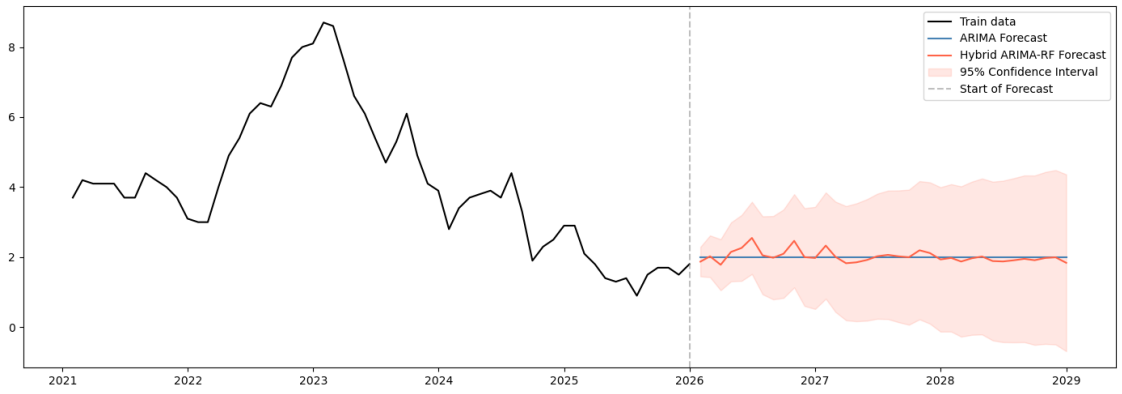
[113]:
```
final_hybrid = final_rf + final_arima_forecast
```

Plot the final Forecast

[115]:
```
last_date = df.index[-1]

forecast_index = pd.date_range(start = last_date, periods = n_forecast+1 ,␣
 ↪freq='MS')[1:]
```

[127]:
```
resid_std = np.std(full_resid)

time_steps = np.arange(1, n_forecast +1)
confidence_interval = 1.96 * resid_std * np.sqrt(time_steps) * 0.3

# Create the upper and lower bounds for your 36-month forecast
hybrid_forecast_df = pd.DataFrame({
    'ARIMA_forecast' : final_arima_forecast.values,
    'Hybrid_forecast' : final_hybrid.values,
    'lower_ci' : final_hybrid.values - confidence_interval,
```

```
    'upper_ci' : final_hybrid.values + confidence_interval
}, index = forecast_index)



fig, ax = plt.subplots(figsize  = (14,5))

ax.plot(df.index, df, label = 'Train data', color = 'black')
ax.plot(hybrid_forecast_df.index, hybrid_forecast_df['ARIMA_forecast'], color =␣
↪'steelblue', label = 'ARIMA Forecast')
ax.plot(hybrid_forecast_df.index, hybrid_forecast_df['Hybrid_forecast'], color␣
↪= 'tomato', label = 'Hybrid ARIMA-RF Forecast')
ax.fill_between(hybrid_forecast_df.index,
                hybrid_forecast_df['upper_ci'],
                hybrid_forecast_df['lower_ci'],
                color = 'tomato',
                label = '95% Confidence Interval',
                alpha = 0.15)

plt.axvline(df.index[-1], color = 'grey', alpha = 0.5, linestyle = '--', label␣
↪= 'Start of Forecast')
plt.tight_layout()
plt.legend()
plt.savefig('forecast_plot.png', dpi = 300, bbox_inches = 'tight')
plt.show()
```



Diagnostics

[124]:
```
# Check 1 - what are the actual RF corrections?
print("RF corrections stats:")
print(f"Mean: {final_rf.mean():.4f}")
print(f"Std: {final_rf.std():.4f}")
print(f"Min: {final_rf.min():.4f}")
```

```
print(f"Max: {final_rf.max():.4f}")

# Check 2 - what's the range of the hybrid vs ARIMA?
print(f"\nARIMA forecast range: {final_arima_forecast.min():.2f} to␣
 ↪{final_arima_forecast.max():.2f}")
print(f"Hybrid forecast range: {final_hybrid.min():.2f} to {final_hybrid.max():.
 ↪2f}")

# Check 3 - confirm which residuals are seeding the history
print(f"\nLast 10 residuals used for seeding:")
print(full_resid[-max(sig_lags):])
```

```
RF corrections stats:
Mean: 0.0190
Std: 0.1659
Min: -0.2181
Max: 0.5480

ARIMA forecast range: 2.00 to 2.00
Hybrid forecast range: 1.78 to 2.55

Last 10 residuals used for seeding:
month
2025-03-31    0.031691
2025-04-30   -0.415206
2025-05-31    0.099225
2025-06-30    0.052389
2025-07-31   -0.525138
2025-08-31    0.851973
2025-09-30   -0.208796
2025-10-31    0.100185
2025-11-30   -0.248071
2025-12-31    0.419030
Freq: M, dtype: float64
```

1.0.1 Conclusion

We were able to capture the non-linearity that the stand alone ARIMA couldn't capture.

We are able to build and fine tune the model with metrics:

1.0.2 Standalone Model:

MSE: 3.1305007701179104

RMSE: 1.7693221216380894

MAE: 1.645753375200405

1.0.3 Hybrid Model:

MSE: 2.9072624743218394

RMSE: 1.7050696391414162

MAE: 1.5855528258983318

[ ]: